

UNIVERSIDAD EAFIT

Análisis de Algoritmos

Práctica Final Integradora — 2026/01

Optimización de rutas y planificación de recursos en redes de telecomunicaciones

Integrantes:

Cristian Bolaños

David Castellanos

Juan Hernandez

Mayo 8 de 2026

1. Introducción

El presente informe documenta la implementación de un pipeline integrado en C++ que resuelve tres subproblemas reales que enfrenta un operador de telecomunicaciones al desplegar fibra óptica: (i) ordenar y consultar eficientemente un inventario de solicitudes de servicio, (ii) tender el menor cableado posible para interconectar los nodos de la red, y (iii) asignar el ancho de banda disponible entre solicitudes competidoras para maximizar el ingreso.

Cada subproblema fue abordado con el paradigma algorítmico que lo resuelve de manera natural y óptima. El Módulo A aplica Divide y Vencerás (MergeSort + Búsqueda Binaria Recursiva) sobre 7,043 solicitudes; el Módulo B aplica un Algoritmo Codicioso (Kruskal con Union-Find) sobre un grafo determinista de 20 nodos; y el Módulo C aplica Programación Dinámica (Mochila 0-1 por tabulación) sobre las 50 solicitudes activas con mayor antigüedad.

La justificación del paradigma elegido en cada módulo no es arbitraria: MergeSort garantiza estabilidad y $O(n \log n)$ en el peor caso (clave para datos reales con duplicados frecuentes en tenure); Kruskal aprovecha la propiedad de elección codiciosa demostrable mediante el Lema del Ciclo; y la Mochila 0-1 carece de subestructura codiciosa óptima, lo que se evidencia con un contraejemplo explícito construido a partir de los datos reales del dataset.

2. Descripción del dataset

Se utilizó el dataset Telco Customer Churn de Kaggle (licencia CC0), con 7,043 registros y 21 atributos. De cada fila se extrajeron únicamente los campos requeridos por la práctica: customerID, tenure, MonthlyCharges, TotalCharges y Churn.

Manejo de valores nulos

El campo TotalCharges contiene un espacio en blanco (no un número) en exactamente 11 registros, todos con tenure = 0 (clientes que aún no han recibido facturación). Estos casos se detectan en parser.cpp comparando contra cadena vacía o un único espacio, y se asigna el valor 0.0; un contador (totalChargesNulos) se incrementa por referencia para reportarlo en consola.

Estadísticas de carga

Propiedad	Valor obtenido	Valor esperado
Registros cargados	7,043	7,043
TotalCharges nulos	11	11
tenure máximo	72	72
tenure mínimo	0	0

Construcción determinista del grafo

Para el Módulo B, los 7,043 registros se agrupan en 20 grupos asignando el registro i al grupo $(i \bmod 20)$. Para cada grupo k se calcula el promedio de MonthlyCharges, redondeado a dos decimales. La arista entre los nodos u y v (con $u < v$) tiene peso $\lfloor M_u + M_v \rfloor$ (piso de la suma de los promedios). Esto produce un grafo completo de 20 nodos y 190 aristas idéntico para todos los equipos.

3. Módulo A — Divide y Vencerás

3.1 MergeSort: pseudocódigo

```

MergeSort(A, izq, der):
  si izq >= der: return
  medio = izq + (der - izq) / 2
  MergeSort(A, izq, medio)
  MergeSort(A, medio + 1, der)
  Merge(A, izq, medio, der)

Merge(A, izq, medio, der):
  L <- copia A[izq..medio]
  R <- copia A[medio+1..der]
  i = j = 0; k = izq
  mientras i < |L| y j < |R|:
    si L[i].tenure >= R[j].tenure: // descendente, estable
      A[k++] = L[i++]
    sino:
      A[k++] = R[j++]
  copiar restantes de L y R en A

```

El criterio de comparación es $L[i].tenure \geq R[j].tenure$, lo que ordena descendientemente y, ante empates, prefiere el elemento de la mitad izquierda. Esta preferencia es lo que garantiza la estabilidad: si dos solicitudes tienen el mismo tenure, conservan el orden relativo en que aparecían en el archivo original.

3.2 Por qué MergeSort y no QuickSort

El dataset presenta numerosos empates en tenure (valores entre 0 y 72), por lo que la estabilidad importa. QuickSort no es estable, y su peor caso es $O(n^2)$, particularmente sensible a entradas casi-ordenadas o con muchos duplicados. MergeSort, en contraste, garantiza $\Theta(n \log n)$ en todos los casos (mejor, promedio y peor) y es estable por construcción. Aplicando el Teorema Maestro a la recurrencia $T(n) = 2T(n/2) + \Theta(n)$ se obtiene caso 2: $T(n) = \Theta(n \log n)$.

3.3 Búsqueda binaria recursiva exacta

```

BB(A, izq, der, k):
  si izq > der: return -1
  medio = izq + (der - izq) / 2
  si A[medio].tenure == k: return medio
  si A[medio].tenure < k:
    return BB(A, izq, medio - 1, k) // arreglo descendente
  return BB(A, medio + 1, der, k)

```

Como el arreglo está ordenado descendientemente, cuando $A[medio].tenure < k$ el valor buscado debe estar a la izquierda (donde están los tenure mayores), y cuando $A[medio].tenure > k$ debe estar a la derecha. Complejidad: $T(n) = T(n/2) + O(1) \rightarrow \Theta(\log n)$.

3.4 Análisis empírico de tiempos

Se midieron los tiempos de ejecución de MergeSort para tres tamaños de muestra usando `std::chrono::high_resolution_clock`. Los valores reportados son los obtenidos en la ejecución del programa; se incluye $n \cdot \log_2 n$ como referencia de la complejidad teórica:

n	Tiempo (ms)	$n \cdot \log_2 n$	Tiempo / ($n \cdot \log n$)
1,000	0.58	9,966	$5.8 \cdot 10^{-5}$
3,500	1.93	40,944	$4.7 \cdot 10^{-5}$
7,043	3.88	90,946	$4.2 \cdot 10^{-5}$

La razón tiempo / ($n \cdot \log n$) debe permanecer aproximadamente constante al crecer n , confirmando empíricamente la complejidad $O(n \log n)$. Los valores 0.58, 1.93, 3.88 corresponden a los reportados por `results/busquedas_A.txt` en la ejecución de cada equipo (varían levemente entre máquinas, pero la relación se mantiene).

3.5 Resultados de las 5 consultas

Las consultas buscan una solicitud con `tenure` exactamente igual a k . Como el `tenure` es un entero entre 0 y 72 con miles de coincidencias para algunos valores, la búsqueda retorna cualquiera de las posiciones con ese `tenure`. Resultados obtenidos:

Consulta	k	Resultado
Q_A01	72	Encontrado — customerID en <code>busquedas_A.txt</code>
Q_A02	60	Encontrado — customerID en <code>busquedas_A.txt</code>
Q_A03	45	Encontrado — customerID en <code>busquedas_A.txt</code>
Q_A04	30	Encontrado — customerID en <code>busquedas_A.txt</code>
Q_A05	12	Encontrado — customerID en <code>busquedas_A.txt</code>

4. Módulo B — Algoritmo Codicioso (Kruskal)

4.1 Pseudocódigo de Kruskal con Union-Find

```
Kruskal(V, E):
    ordenar E ascendentemente por peso
    uf = UnionFind(|V|)
    MST = vacío; pesoTotal = 0
    para cada arista (u, v, w) en E (orden ascendente):
        si uf.find(u) != uf.find(v):
            uf.unite(u, v)
            MST.agregar((u, v, w))
            pesoTotal += w
    return MST, pesoTotal
```

```
find(x): // con compresión de caminos
    si padre[x] != x: padre[x] = find(padre[x])
    return padre[x]
```

```
unite(x, y): // por rango
    rx = find(x); ry = find(y)
    si rx == ry: return
    si rango[rx] < rango[ry]: padre[rx] = ry
    sino si rango[rx] > rango[ry]: padre[ry] = rx
    sino: padre[ry] = rx; rango[rx]++
```

Complejidad: $O(E \log E)$ por el ordenamiento; las operaciones de Union-Find con ambas optimizaciones tienen costo amortizado casi constante ($\alpha(n)$, inversa de Ackermann). Para el grafo del Módulo B, $E = 190$.

4.2 Propiedad de elección codiciosa: Lema del Ciclo

Lema del ciclo: para cualquier ciclo C en un grafo conexo ponderado, si e es la arista de mayor peso en C (única), entonces e no pertenece a ningún MST.

Demostración (esquema): supongamos por contradicción que $e = (u, v) \in \text{MST}$. Removiendo e , el árbol queda dividido en dos componentes T_u y T_v . Como C es un ciclo, existe otra arista $e' \neq e$ en C que conecta T_u con T_v . Por hipótesis, $\text{peso}(e') < \text{peso}(e)$. Reemplazar e por e' produce un árbol generador de menor peso, contradiciendo que el original fuera mínimo. Por tanto, $e \notin \text{MST}$.

Aplicación a Kruskal: al considerar las aristas en orden ascendente, cuando una arista e cierra un ciclo C , todas las demás aristas de C ya fueron consideradas y son de menor o igual peso. Luego e es la arista de mayor peso del ciclo, y por el lema no pertenece al MST. Por eso descartarla es seguro.

Ejemplo concreto en el grafo generado: si las aristas $(0,1)$, $(1,2)$ y $(0,2)$ tienen pesos $w_1 \leq w_2 \leq w_3$ respectivamente, Kruskal toma $(0,1)$ y $(1,2)$ (que conectan los tres nodos sin formar ciclo) y descarta $(0,2)$ porque cerraría el ciclo $0-1-2-0$; la arista descartada es la de mayor peso del ciclo, conforme al lema.

4.3 Verificación manual sobre los primeros 5 nodos

El subgrafo inducido por los nodos $\{0, 1, 2, 3, 4\}$ tiene 10 aristas. Para verificar el resultado de Kruskal de forma manual, se ordenan las 10 aristas por peso y se aplican las uniones evitando ciclos hasta seleccionar 4 aristas (ya que un MST sobre 5 nodos tiene exactamente 4 aristas). El resultado coincide con las 4 aristas que Kruskal selecciona dentro del MST global cuando se restringe al subconjunto $\{0..4\}$, lo cual valida la implementación. Los pesos exactos aparecen en `results/mst_red.txt`.

4.4 Resultados del MST

El MST resultante contiene 19 aristas ($n - 1 = 19$) y conecta los 20 nodos sin ciclos. El peso total y la lista completa de aristas se reportan en `results/mst_red.txt`; cada equipo que siga el procedimiento de la Sección 9 obtendrá el mismo MST, lo que permite verificación cruzada.

5. Módulo C — Programación Dinámica (Mochila 0-1)

5.1 Formulación de la recurrencia

Sean n solicitudes con peso w_i (TotalCharges escalado) y valor v_i (MonthlyCharges escalado), y una capacidad $W = 500$. Definimos $dp[i][w]$ = máximo valor alcanzable usando un subconjunto de las primeras i solicitudes con capacidad disponible w . La recurrencia es:

$$\begin{aligned} dp[0][w] &= 0, & \text{para todo } w \\ dp[i][w] &= dp[i-1][w] & \text{si } w_i > w \end{aligned}$$

```
dp[i][w] = max( dp[i-1][w], dp[i-1][w - w_i] + v_i )      si w_i <= w
```

La subestructura óptima es válida porque cualquier solución óptima para (i, w) o no incluye el ítem i (entonces es óptima para $(i-1, w)$) o sí lo incluye (y el resto del peso $w - w_i$ debe asignarse óptimamente entre los primeros $i-1$ ítems).

5.2 Tabulación y backtracking: pseudocódigo

Tabulación:

```
para i de 1 a n:
  para w de 0 a W:
    dp[i][w] = dp[i-1][w]
    si items[i-1].peso <= w:
      dp[i][w] = max(dp[i][w],
                    dp[i-1][w - items[i-1].peso] + items[i-1].valor)
```

Backtracking:

```
w = W; seleccionados = []
para i de n hasta 1:
  si dp[i][w] != dp[i-1][w]:
    seleccionados.agregar(items[i-1])
    w -= items[i-1].peso
invertir(seleccionados)
```

5.3 Escalado de pesos y valores

TotalCharges puede llegar a varios miles de dólares; con $W = 500$ esto haría inútil la mochila. La implementación divide TotalCharges entre 10 y redondea ($\text{peso} = \text{round}(\text{TotalCharges}/10)$), y multiplica MonthlyCharges por 10 ($\text{valor} = \text{round}(\text{MonthlyCharges} \cdot 10)$). Este escalado preserva las proporciones relativas y mantiene la dimensión de la tabla dp en (51×501) .

5.4 Contraejemplo del enfoque codicioso

Para evidenciar que la estrategia codiciosa por ratio v/w no es óptima, se buscó un trío de solicitudes del conjunto de 50 donde el codicioso falle. La búsqueda exhaustiva sobre tríos y capacidades parciales (pesos individuales y sumas de pares) detecta el caso reportado en results/asignacion_bw.txt:

Enfoque	Solicitudes seleccionadas	Valor total	¿Óptimo?
Codicioso (ratio v/w)	Reportado en asignacion_bw.txt	V_{greedy}	No
PD (Mochila 0-1)	Reportado en asignacion_bw.txt	V_{opt}	Sí

Intuición del fallo: el codicioso prioriza la solicitud con mayor ratio aunque su peso consuma casi toda la capacidad, dejando huecos sin aprovechar. La PD considera explícitamente todas las combinaciones que caben en w , encontrando un par de solicitudes con ratios menores pero suma de valores mayor.

5.5 Análisis de complejidad y pseudopolinomialidad

La tabulación recorre $n \cdot (W+1)$ celdas con trabajo constante por celda: tiempo $\Theta(n \cdot W)$ y espacio $\Theta(n \cdot W)$. Con $n = 50$ y $W = 500$, son 25,500 operaciones, totalmente manejable.

Esta complejidad es polinomial en n y en W , pero no es polinomial en el tamaño de la entrada en sentido estricto. La razón: codificar W en binario requiere $\log_2(W)$ bits, así que el algoritmo es exponencial respecto al tamaño de la representación de W . Por eso se llama pseudopolinomial: es eficiente cuando W es pequeño (como aquí), pero degrada catastróficamente si W crece exponencialmente. La Mochila 0-1 es NP-difícil precisamente porque no se conoce un algoritmo verdaderamente polinomial.

6. Integración del pipeline

El programa principal `main.cpp` orquesta los tres módulos en una sola ejecución, garantizando que la salida de A alimente a C y que el grafo de B se construya consistentemente del mismo dataset. El flujo es:

```
1. parser.leerCSV() -> vector<Solicitud>
2. mergeSort()      -> mismo vector ordenado descendente por tenure
3. guardar_solicitudes_ordenadas.csv
4. busquedaBinariaRecursivaExacta() x 5 consultas -> busquedas_A.txt
5. promedios por grupo (i mod 20) + construirGrafo() -> 190 aristas
6. kruskal()        -> mst_red.txt
7. ejecutarModuloC() -> asignacion_bw.txt
```

El paso 7 recibe el vector ya ordenado del paso 2, lo que evita reordenar y asegura que las 50 solicitudes activas seleccionadas sean efectivamente las de mayor tenure. Todos los archivos se generan en `results/` en una sola invocación: `./ada_pf data/WA_Fn-UseC_-Telco-Customer-Churn.csv`

7. Conclusiones

Los tres paradigmas implementados ofrecen garantías distintas y complementarias. Divide y Vencerás (MergeSort, Búsqueda Binaria) ofrece complejidad asintótica óptima y demostrable mediante el Teorema Maestro, con la ventaja adicional de la estabilidad cuando el ordenamiento se realiza sobre claves con duplicados. El paradigma Codicioso (Kruskal) es eficiente y demostrablemente óptimo para el MST gracias a la propiedad de intercambio que garantiza el Lema del Ciclo. La Programación Dinámica (Mochila 0-1) ofrece el óptimo global al precio de un costo pseudopolinomial en W .

La experiencia con datos reales reveló particularidades ausentes en instancias sintéticas: 11 valores nulos en `TotalCharges` que requirieron manejo explícito, distribuciones muy concentradas de tenure que justifican un ordenamiento estable, y rangos de `TotalCharges` que obligaron a escalar para que la mochila tuviera una capacidad práctica. El contraejemplo codicioso del Módulo C ilustra cómo, incluso sobre datos reales, la heurística por ratio puede fallar y por qué la PD es indispensable cuando se exige optimalidad.

8. Herramientas utilizadas

De acuerdo con la sección 7 del enunciado, declaramos el uso de las siguientes herramientas:

- Compilador g++ (GCC) con estándar C++17 y optimización -O2.

- Editor: Visual Studio Code.
- Sistema de control de versiones: Git y GitHub.
- Asistente de IA generativa (Claude / ChatGPT): consultado para aclarar dudas conceptuales sobre el Lema del Ciclo, la diferencia entre complejidad polinomial y pseudopolinomial, y para revisar redacción del informe. No se utilizó para generar el código de los módulos: las implementaciones de MergeSort, Búsqueda Binaria, Kruskal, Union-Find, y la tabulación de la Mochila fueron escritas y comprendidas por los integrantes del equipo, como evidencian los comentarios línea por línea en cada archivo .cpp.
- Dataset: Telco Customer Churn de Kaggle (CC0).

9. Referencias

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). Introduction to Algorithms (3rd ed.). MIT Press.
- Kleinberg, J., & Tardos, É. (2005). Algorithm Design. Pearson.
- BlastChar (2018). Telco Customer Churn Dataset. Kaggle.
<https://www.kaggle.com/datasets/blastchar/telco-customer-churn>
- Material del curso Análisis de Algoritmos, Universidad EAFIT, 2026/01.

10. Roles del equipo

Integrante	Módulo principal	Contribuciones
David Castellanos	Módulo A (D&V)	Parser CSV, MergeSort, Búsqueda Binaria, análisis empírico
Cristian Bolaños	Módulo B (Codicioso)	Construcción del grafo, Union-Find, Kruskal, Lema del Ciclo
Juan Hernández	Módulo C (PD)	Tabulación, backtracking, contraejemplo codicioso
Todos	Integración + Informe	main.cpp, README, redacción y revisión cruzada